MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Report

on laboratory work no. 4

of Cryptography and Security

Topic: Block Ciphers. The DES Algorithm

Done by: st. gr. FAF-243, Augustin Carazanu
Checked by: *assist. univ. Maia Zaica*
FCIM, UTM

Chişinău – 2026

Purpose of the work

The purpose of this laboratory work is to study block ciphers and the DES (Data Encryption Standard) algorithm from the course materials, and to implement one element of DES in a program. The task is chosen with the formula nr_task = n mod 11. My number in the group list is n = 3, so 3 mod 11 = 3, and I solved **Task 2.3: in the DES algorithm K+ is given; determine the round key Ki for a given i**. As required, the program displays the tables used and all the intermediate steps, and the input data can be entered by the user or generated randomly.

The theory below follows the materials of the course: the lecture “Fundamente ale Criptografiei Moderne. Tema 4. Cifruri de tip bloc” (A. Zgureanu), the slides “4. Block ciphers”, the standards FIPS 46 and FIPS 46-3, the “DES Tables” document and the “ASCII Character Set” table.

1. Short theory

1.1. Block ciphers and the Feistel network

A block cipher encrypts the message in blocks of n bits, using a mathematical function of the plaintext block and the key, and produces a ciphertext block of the same size. Knowing the key, encryption and decryption must be fast, while knowing pairs of plaintext and ciphertext blocks it must be hard to find the key.

Many block ciphers, including DES, are built on the **Feistel network** (Horst Feistel, 1971). The block is split into two halves L0 and R0, and in round i:

$$Li = Ri-1, \quad Ri = Li-1 \oplus f(Ri-1, Ki)$$

where ⊕ is XOR, f is the round function and Ki is the round key. Whatever f is, decryption uses the same structure: Ri−1 = Li and Li−1 = Ri ⊕ f(Li, Ki), with the round keys taken in reverse order. The rounds combine permutations (P-boxes), which create **diffusion**, and non-linear substitutions (S-boxes), which create **confusion**, as described by Claude Shannon.

1.2. The DES algorithm

DES was developed by IBM from the Lucifer cipher, adopted as a U.S. federal standard on 23 November 1976 and published in 1977 (FIPS 46; the last revision is FIPS 46-3 from 1999). Its main parameters are:

• block size: **64 bits**; key: **64 bits**, of which **56** are effective and 8 are parity bits;

• **16 identical rounds**, each with a different 48-bit round key derived from the main key;

• the same key is used for encryption and decryption (symmetric cipher).

Encryption of a 64-bit block has three stages: the **initial permutation IP**, after which the block is split into L0 and R0 (32 bits each); **16 Feistel rounds**; and, after swapping the halves (R16L16), the **final permutation IP^{-1}**, which is the inverse of IP. Bits are numbered from left to right, starting with 1, as in FIPS 46-3.

1.3. The round function f

The function f(Ri−1, Ki) has four steps:

• **Expansion E**: the 32-bit half is expanded to 48 bits (some bits are repeated), which increases diffusion;

• **XOR with the round key**: E(Ri−1) ⊕ Ki, 48 bits;

• **S-boxes**: the 48 bits are split into eight 6-bit groups B1…B8, and each group is replaced by 4 bits using S1…S8. The outer bits (first and last) give the row and the four inner bits give the column. For example, in S5 the input 011011 has the row 01 = 1 and the column 1101 = 13, so the output is 9 = 1001. The S-boxes are the only non-linear part of DES and are essential for its security;

• **Permutation P** of the resulting 32 bits.

The result is XORed with Li−1 to give Ri.

1.4. The key schedule (the subject of Task 2.3)

The key schedule KS derives the 16 round keys Kn = KS(n, KEY) of 48 bits from the 64-bit key:

a) PC-1. The 64-bit key K is permuted with Permuted Choice 1 (Table 1.1). PC-1 removes the parity bits 8, 16, 24, 32, 40, 48, 56 and 64 and gives the 56-bit key **K+**. The first four rows of PC-1 select the bits of C0, the last four the bits of D0.

Table 1.1 – PC-1: position in K of each bit of K+

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

b) Split. K+ is split into two halves of 28 bits: **C0** (bits 1–28) and **D0** (bits 29–56).

c) Left shifts. For n = 1 … 16, Cn = LSn(Cn−1) and Dn = LSn(Dn−1), where LSn is a circular left shift by 1 position in rounds 1, 2, 9 and 16 and by 2 positions in all other rounds (Table 1.2). The total is 4 × 1 + 12 × 2 = 28 shifts, so **C16 = C0** and **D16 = D0**.

Table 1.2 – Number of left shifts in each round

Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Shifts	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

d) PC-2. The halves are joined into CnDn (56 bits) and Permuted Choice 2 selects 48 of them, in the order of Table 1.3. The first bit of Kn is bit 14 of CnDn, the second is bit 17, and so on. Bits 9, 18, 22, 25, 35, 38, 43 and 54 are not used. The first 24 bits of Kn are taken from Cn and the last 24 from Dn.

Table 1.3 – PC-2: position in CnDn of each bit of Kn

14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

For **decryption** the same round keys are used in reverse order (K16 first). Because C16 = C0 and D16 = D0, they can be generated on the fly: the first decryption key is obtained by applying PC-2 directly after PC-1, and the halves are then rotated to the **right** (by 1 position in rounds 2, 9 and 16 and by 2 positions in the other rounds).

1.5. Security of DES

DES resists differential and linear cryptanalysis well (linear cryptanalysis needs 2^{43} known plaintext–ciphertext pairs). Its weakness is the short key: $2^{56} \approx 7.2 \times 10^{16}$ keys can be searched exhaustively, and in 1998 the machine Deep Crack found a DES key in 56 hours. For this reason DES was strengthened as 3DES (C = DES(k3, DES^{-1}(k2, DES(k1, M)))) and replaced by AES in 2002; it was officially withdrawn in 2005.

2. The task

Task 2.3 (n = 3, 3 mod 11 = 3). In the DES algorithm K+ is given. Determine the round key Ki for a given i.

Input: K+ (56 bits) and the round number i (1 … 16). According to the lab sheet, K+ must be possible to enter by the user or to generate randomly, and the program must show the tables used and every intermediate step. As an extra option, the program can also compute K+ from an 8-character key, converting the characters with the ASCII table and applying PC-1, as in exercise 13 of the lecture.

3. Implementation

The program has two files: `des_keys.py` contains the tables (copied from the “DES Tables” document and checked against FIPS 46-3) and the key-schedule functions, and `gui.py` contains the Tkinter window. K+ can be typed as 56 binary digits or 14 hexadecimal digits, generated randomly with the button “Random K+”, or computed from an 8-character key. The round i is chosen from 1 to 16.

3.1. The tables

```
PC2 = [14, 17, 11, 24, 1, 5,
       3, 28, 15, 6, 21, 10,
       23, 19, 12, 4, 26, 8,
       16, 7, 27, 20, 13, 2,
       41, 52, 31, 37, 47, 55,
       30, 40, 51, 45, 33, 48,
       44, 49, 39, 56, 34, 53,
       46, 42, 50, 36, 29, 32]

# Number of left shifts for rounds 1..16
SHIFTS = [1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1]
```

Positions are numbered from 1, as in the standard.

3.2. Input validation

```
def parse_kplus(text):
    """K+ given as 56 binary digits or 14 hex digits (spaces allowed)."""
    s = "".join(text.split())
    if not s:
        raise ValueError("Enter K+ (56 bits in binary or 14 hexadecimal digits).")
    if set(s) <= {"0", "1"} and len(s) != 14:
        if len(s) != 56:
            raise ValueError(f"K+ in binary must have exactly 56 bits (it has
{len(s)}).")
        return s
    if all(c in "0123456789abcdefABCDEF" for c in s):
        if len(s) != 14:
            raise ValueError(f"K+ in hexadecimal must have exactly 14 digits (it has
{len(s)}).")
        return bin(int(s, 16))[2:].zfill(56)
    bad = next(c for c in s if c not in "0123456789abcdefABCDEF")
    raise ValueError(f"Invalid character '{bad}'. K+ may contain only 0/1 (56 bits) "
                     f"or hexadecimal digits 0-9, A-F (14 digits).")


def parse_round(value):
    v = str(value).strip()
    if not v.isdigit() or not 1 <= int(v) <= 16:
        raise ValueError(f"Invalid round '{v}'. i must be an integer from 1 to 16.")
    return int(v)
```

`parse_kplus()` removes the spaces and accepts K+ either as exactly 56 bits or as exactly 14 hexadecimal digits; otherwise it explains the correct format. `parse_round()` accepts only an integer from 1 to 16.

3.3. Permutation and circular left shift

```
def permute(bits, table):
    return "".join(bits[p - 1] for p in table)


def left_shift(bits, n):
    return bits[n:] + bits[:n]
```

`permute()` takes, for each entry of the table, the bit at that position (minus 1, because Python counts from 0). `left_shift()` moves the first n bits to the end, which is the circular left shift LSn.

3.4. Computing Ki

```
def round_key(kplus, i):
    """Return K_i and every intermediate step needed to obtain it."""
    c, d = kplus[:28], kplus[28:]
    steps = [(0, 0, c, d)]
    for n in range(1, i + 1):
        c, d = left_shift(c, SHIFTS[n - 1]), left_shift(d, SHIFTS[n - 1])
        steps.append((n, SHIFTS[n - 1], c, d))
    cd = c + d
    return {"C0": kplus[:28], "D0": kplus[28:], "steps": steps,
            "CD": cd, "K": permute(cd, PC2)}


# ------------------------------------------------------------------
```

`round_key()` splits K+ into C0 and D0, applies the shifts LS1 … LSi and keeps every pair (Cn, Dn) so that the interface can display it. Then it joins Ci and Di and applies PC-2. In the window, step 4 also shows, for each bit of Ki, its position in CiDi and its value, so the result can be checked bit by bit.

3.5. K+ from an 8-character key and random K+

```
def random_kplus():
    return "".join(secrets.choice("01") for _ in range(56))


def key_to_bits(key):
    """8 ASCII characters -> 64 bits."""
    if len(key) != 8:
        raise ValueError(f"The key must have exactly 8 characters (it has
{len(key)}).")
    if any(ord(c) > 127 for c in key):
        raise ValueError("The key must contain only ASCII characters.")
    return "".join(format(ord(c), "08b") for c in key)


# ---------------------------------------------------------------
# Key schedule
# ---------------------------------------------------------------
def permute(bits, table):
    return "".join(bits[p - 1] for p in table)


def left_shift(bits, n):
    return bits[n:] + bits[:n]


def kplus_from_key(key):
    k64 = key_to_bits(key)
    return k64, permute(k64, PC1)
```

`key_to_bits()` converts each character into 8 bits using its ASCII code, and `kplus_from_key()` applies PC-1. `random_kplus()` generates 56 random bits.

4. Results

4.1. Exercise 14 from the lecture: k1 for a given K+

The lecture asks: given K+ = 1011100 1010101 0110101 1101101 1010010 1010100 0110001 1011010, determine the round key k1. Solution by hand:

Step 1 – split K+:

```
C0 = 1011100 1010101 0110101 1101101
D0 = 1010010 1010100 0110001 1011010
```

Step 2 – LS1 (round 1 has 1 shift): the first bit of each half moves to the end.

```
C1 = 0111001 0101010 1101011 1011011
D1 = 0100101 0101000 1100011 0110101
```

Step 3 – PC-2 on C1D1: the first group of k1 is made of bits 14, 17, 11, 24, 1 and 5 of C1D1. These are 0, 0, 1, 1, 0, 0, so the first group is 001100. Continuing in the same way with all the rows of PC-2:

```
k1 = 001100 111010 000100 111111 010100 101011 111100 000000
   = 33A13F52BF00 (hex)
```

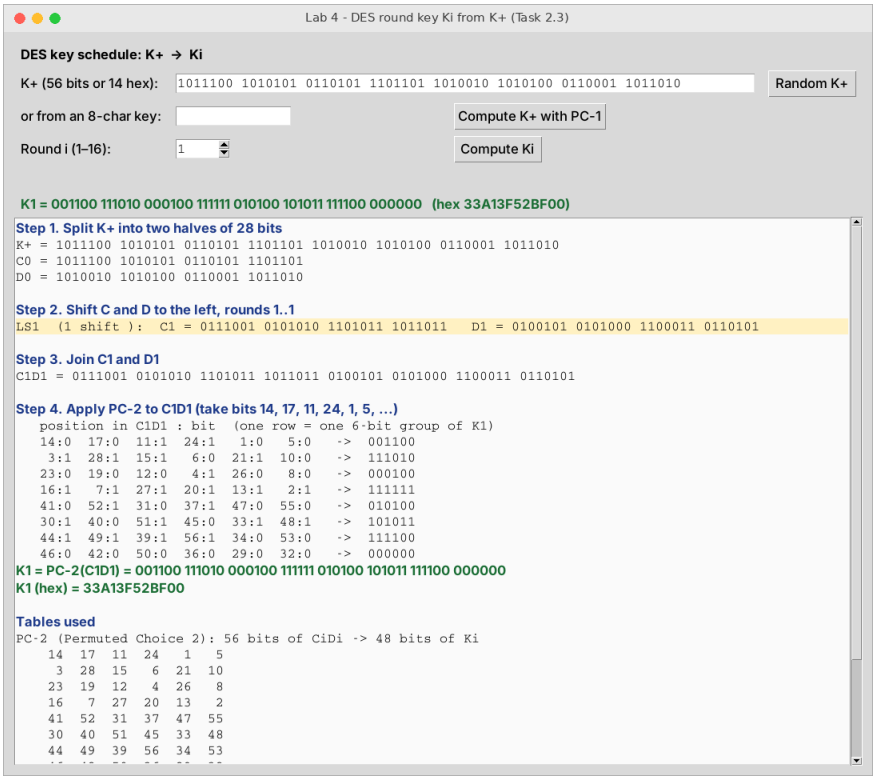


Figure 4.1 – Exercise 14: k1 computed by the program, with every bit of PC-2

The program gives the same result. In step 4 every row shows the six positions of a PC-2 row, the bit found at each position and the resulting 6-bit group.

4.2. Exercise 13 from the lecture: K+ from the key *L&d1$vV

Each character is converted to 8 bits using the ASCII table (for example * = 2A = 00101010, L = 4C = 01001100), which gives the 64-bit key K. PC-1 drops the parity bits and gives:

```
K  = 00101010 01001100 00100110 01100100 00110001 00100100 01110110 01010110
K+ = 0000000 0110010 1001111 1011101 1100010 1111011 1000000 0110000 = 00CA7DDC5EE030
```

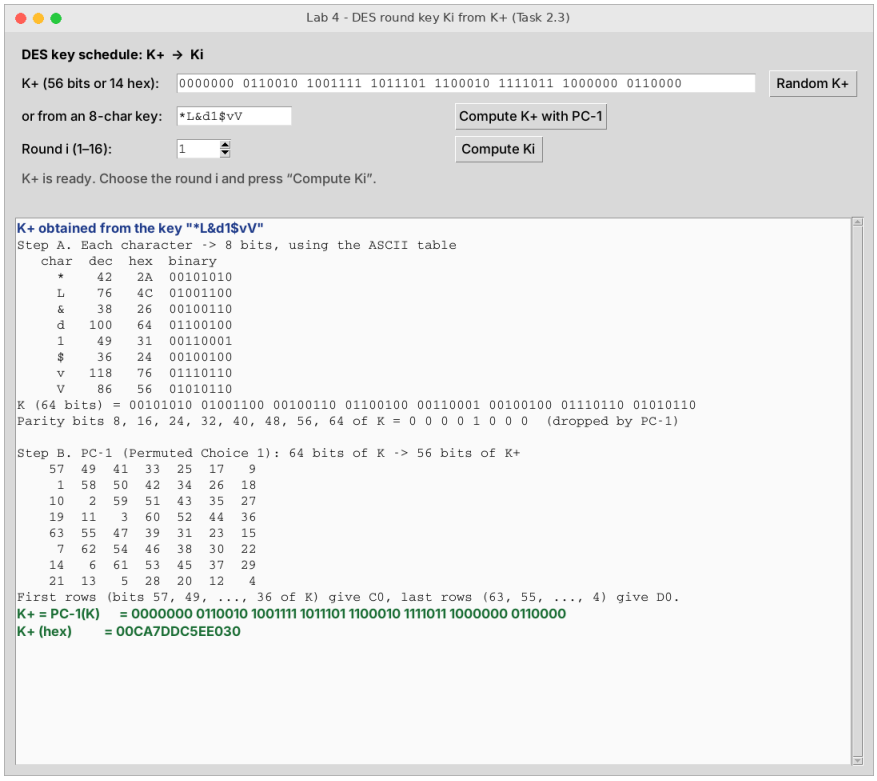


Figure 4.2 – Exercise 13: the ASCII conversion of the key and PC-1

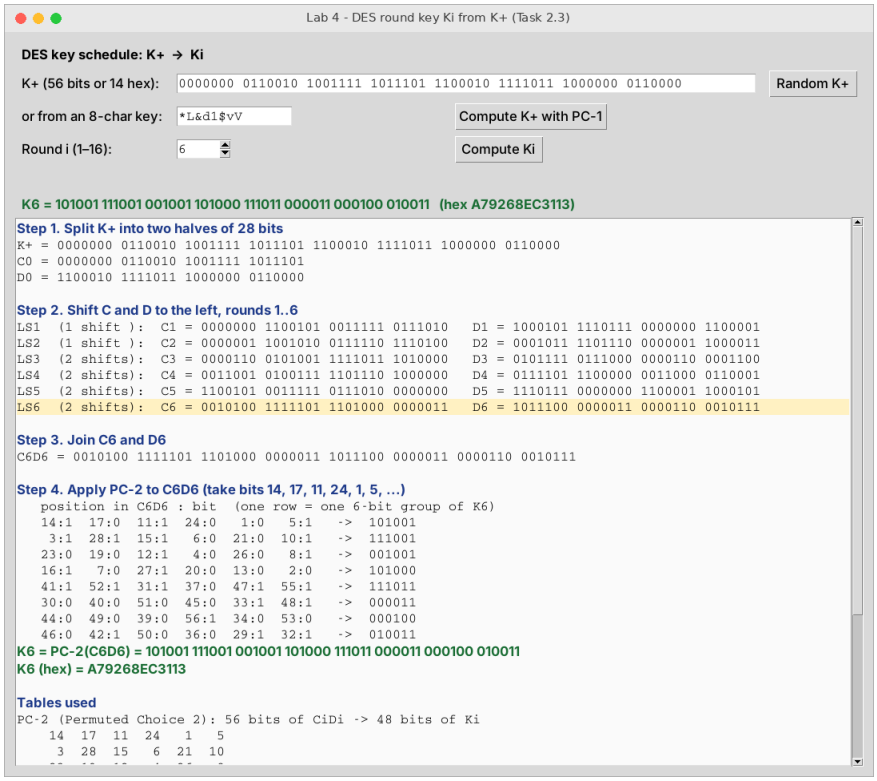


Figure 4.3 – K6 for the K+ of exercise 13

4.3. Check against the standard example

To check the program against independent data, I used the key K = 133457799BBCDFF1 from the well-known illustrated example of DES (J. O. Grabbe), whose round keys are published. Its K+ is F0CCAAF556678F.

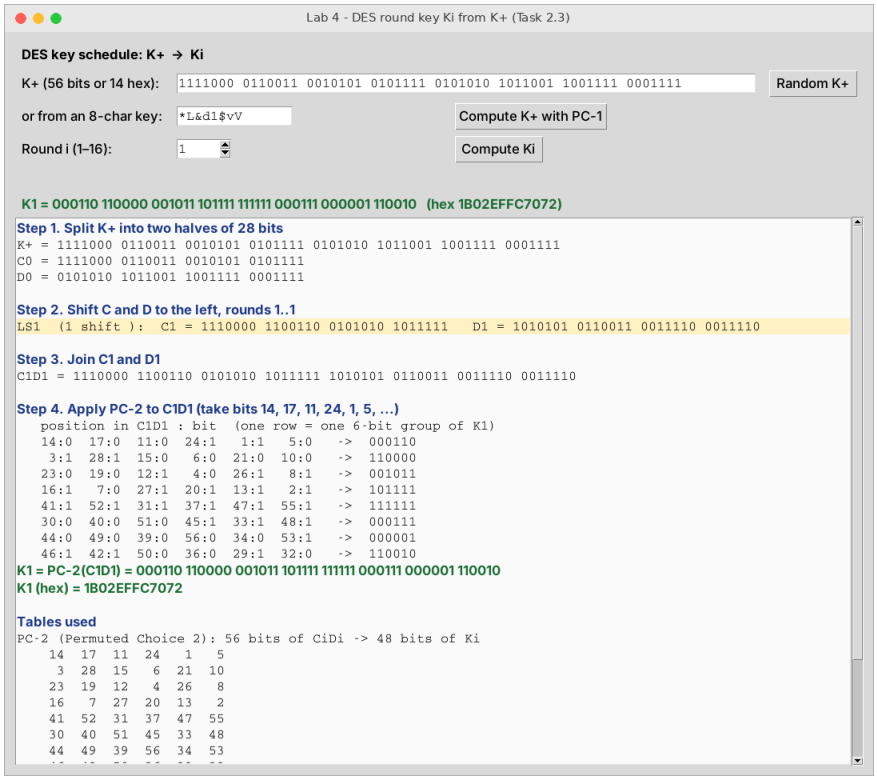


Figure 4.4 – K1 for the standard example

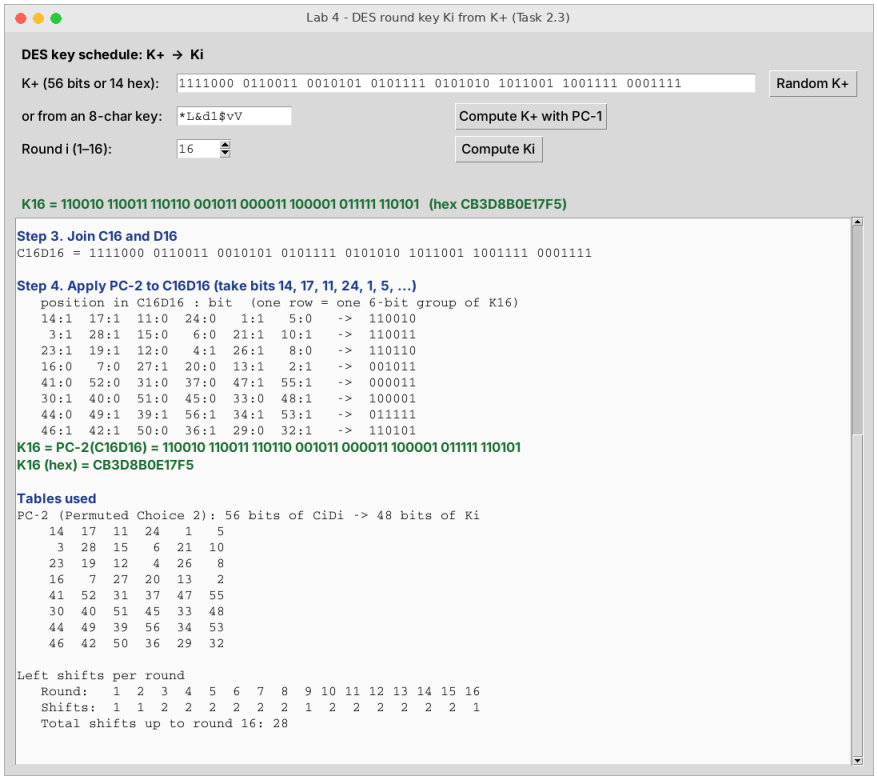


Figure 4.5 – K16: after 28 shifts in total, C16 = C0 and D16 = D0

K1 = 000110 110000 001011 101111 111111 000111 000001 110010 and K16 = 110010 110011 110110 001011 000011 100001 011111 110101 are the same as the published values. Figure 4.5 also confirms that C16 = C0 and D16 = D0.

4.4. Random K+

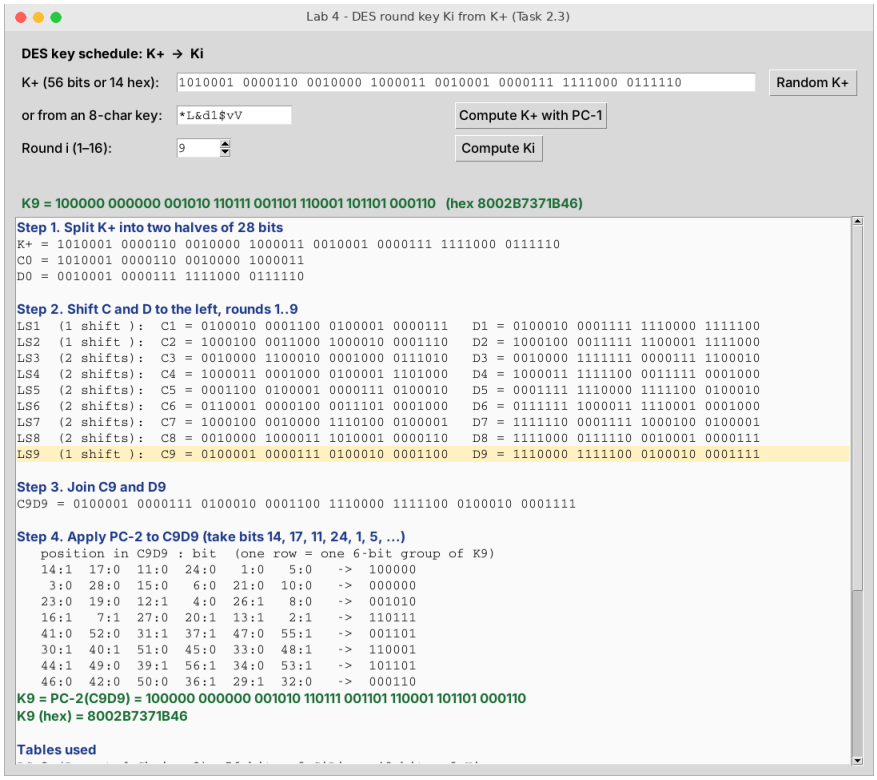


Figure 4.6 – K9 for a randomly generated K+

4.5. Invalid input

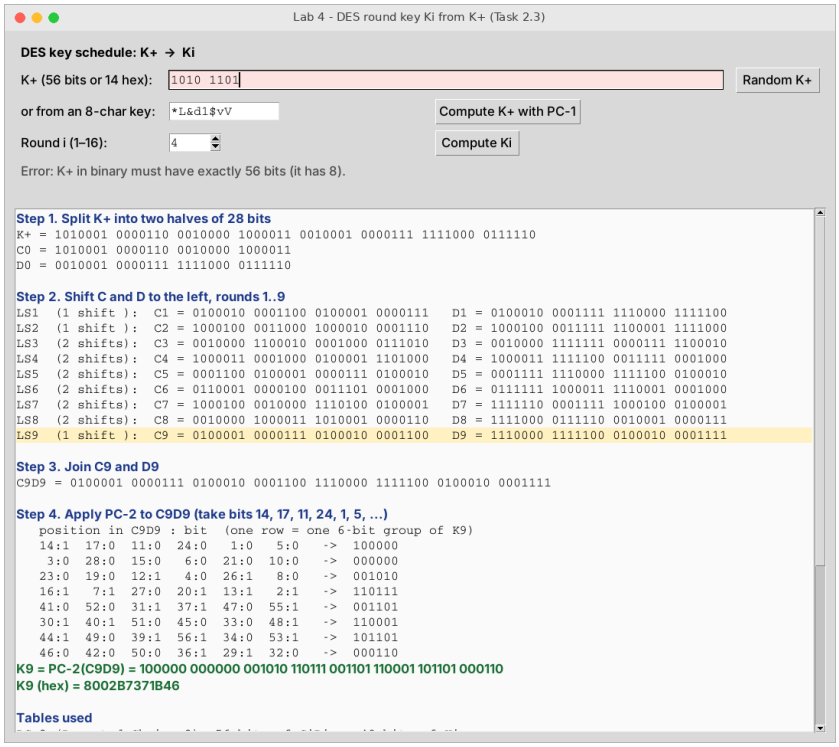


Figure 4.7 – K+ with a wrong number of bits

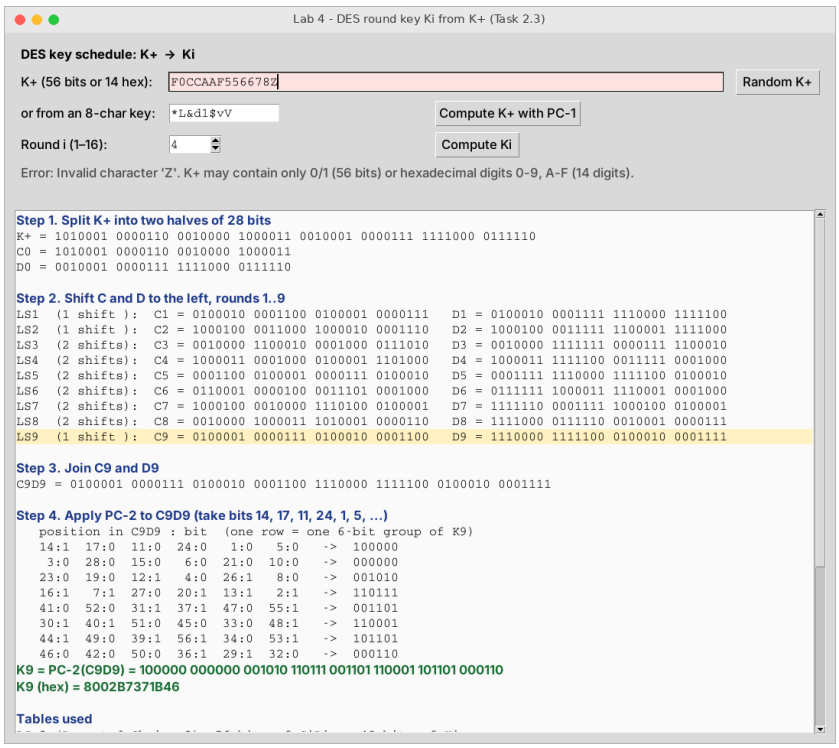


Figure 4.8 – K+ with an invalid character

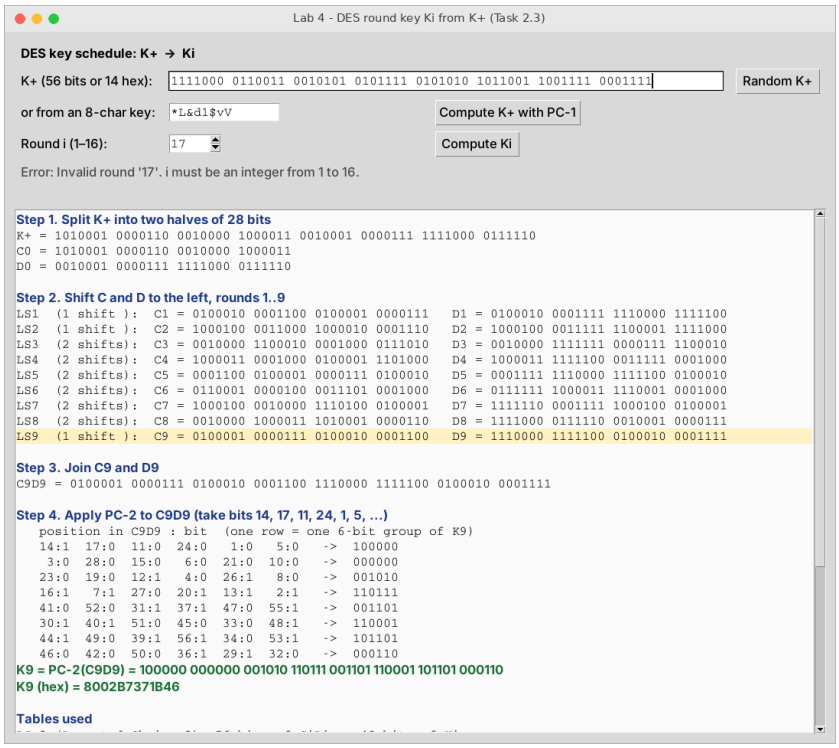


Figure 4.9 – A round number outside 1…16

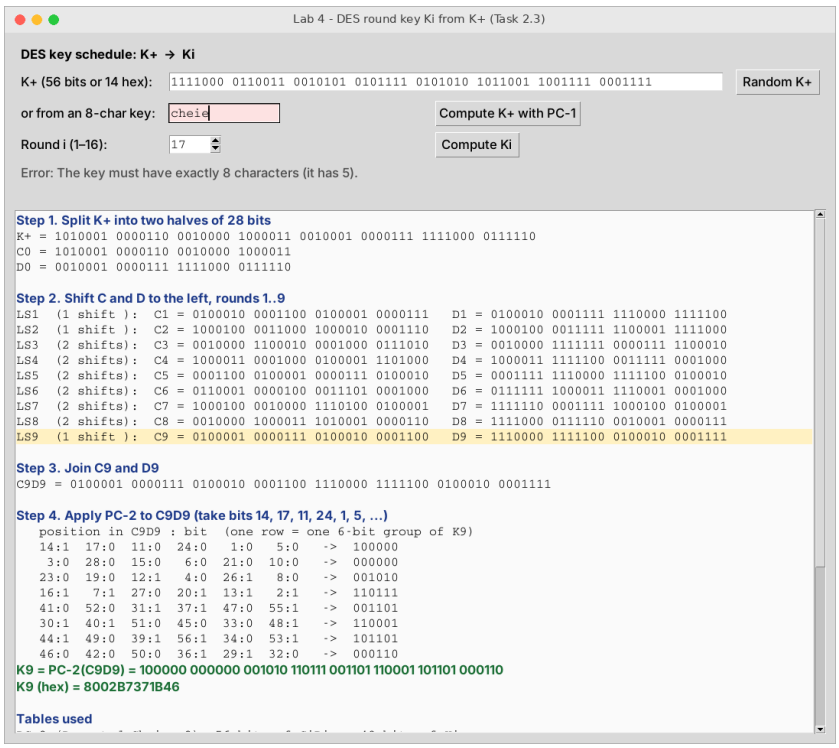


Figure 4.10 – A key that does not have 8 characters

5. Conclusions

DES is a 16-round Feistel cipher in which every round uses its own 48-bit key. The key schedule is simple: PC-1 removes the 8 parity bits and gives K+, the two 28-bit halves are rotated to the left by 1 or 2 positions in each round, and PC-2 selects 48 of the 56 bits. Because the shifts add up to 28, the halves return to C0 and D0 after round 16, which allows the keys to be generated in reverse order for decryption by rotating to the right.

Every round key uses a different selection of the key bits, so the influence of each key bit is spread over almost all the rounds. Together with the S-boxes (confusion) and the permutations E and P (diffusion), this makes DES resistant to analytical attacks. Its real weakness is the 56-bit key, which can be found by exhaustive search; this is why it was replaced by 3DES and AES.

The program solves Task 2.3 and shows every intermediate step: C0 and D0, all the shifts up to round i, CiDi and the bit-by-bit application of PC-2. Its results agree with the hand calculation of exercise 14 from the lecture and with the published round keys of the standard example.

6. Bibliography

1. A. Zgureanu, Fundamente ale Criptografiei Moderne. Tema 4. Cifruri de tip bloc.

2. A. Zgureanu, Cryptography & Security. 4. Block ciphers (course slides).

3. FIPS PUB 46, Data Encryption Standard (DES), NBS, 1977.

4. FIPS PUB 46-3, Data Encryption Standard (DES), NIST, 1999.

5. DES Tables; ASCII Character Set (course materials on ELSE).

7. Source code

The full source code is available on GitHub: https://github.com/Afyze/cs-2026-AugustinCarazanu/tree/main/lab4